

Software Engineering Management

ACRONYMS

MBSE	Model-Based System Engineering
<i>PMBOK® Guide</i>	<i>Guide to the Project Management Body of Knowledge</i>
PSM	Practical Software and Systems Measurement
SDLC	Software development life cycle
SEM	Software engineering management
SQA	Software quality assurance
SWX	<i>Software Extension to the PMBOK® Guide</i>
WBS	Work breakdown structure

INTRODUCTION

Software engineering management (SEM) can be defined as a collection of work activities involved with planning, estimating, measuring, controlling, coordinating, leading and managing risk factors for a software project to help ensure that software products and software engineering services are delivered efficiently, effectively and to the stakeholders' benefit [3]. Although project management and measurement management are often seen as separate areas, and each possesses many unique attributes, the close relationship between the two has led to their combined treatment in this knowledge area (KA).

In one sense, it should be possible to manage a software engineering project in the same way other complex endeavors are managed, using models, technical processes and problem-solving styles as other engineering projects do. However, software engineers use

different process models, technical processes, and problem-solving styles than other engineers, making these choices based on their education and experience and on the differences between physical and software attributes. Software system elements are logical constructions expressed in algorithmic form, while physical system elements are realized in mechanical, electrical, chemical, biological and other physical media. Software is intangible because it has no physical properties and is malleable because of the relative ease with which code can be modified. Obtaining the desired effect by modifying software code might not be easy, but code modifications, per se, are straightforward compared with the modification of physical elements that have already been constructed [12].

As software and software-embedded systems become bigger, more complex, and more intertwined, software engineering management and engineering roles are evolving in response [10], because skilled individuals must actively develop and maintain these systems. Consider the following: hardware is different from software (and not all software is the same). Hardware can be developed, procured, and maintained in a linear fashion. Software is an enduring capability that must be supported and continuously improved throughout its life cycle [13]. Furthermore, the malleable nature of software allows iteration among and interleaving of the development phases (to a much greater degree than is possible when developing physical artifacts).

Software is made by people and for people, so digital talent matters. Software projects are increasingly important, and their ongoing success largely depends on people with the right skills, knowledge and abilities. This fact

is essentially actual and necessary but not sufficient. Other human factors may affect the project's success. During the software development lifecycle, it is impossible to separate the human factors from the technical ones. Therefore, people management activities, such as team and teamwork, leadership, communication, and coordination activities, are important to project success.

Software reuse can be a key factor in maintaining and improving productivity and competitiveness.

Factors such as cultural differences and diverse attitudes may affect the development team. A significant number of software projects failed due to social issues. A "high quality" developer can produce inappropriate or poor quality products that require rework if presented with poor requirements or communication.

Other issues can complicate effective management of software projects and software life cycle processes, including the following:

- Clients often do not know what is needed or what is feasible.
- Increased understanding and changing conditions will likely generate new or changed software requirements.
- Clients often do not appreciate the complexities inherent in software engineering, particularly regarding the impact of changing requirements.
- As a result of changing requirements and software malleability, software is often built iteratively rather than as a linear sequence of phases.
- Software is nominally an enduring capability that must be supported and continuously improved throughout its lifecycle.
- Software construction differs from hardware implementation in that design is usually part of software construction, whereas in hardware-oriented systems, design precedes hardware implementation to "get it right" prior to procurement or fabrication of hardware [12].
- Software engineering necessarily incorporates creativity and discipline. Maintaining

an appropriate balance between the two is sometimes difficult [5].

- The development of software capabilities often involves a high degree of novelty and complexity.
- Typically, the underlying technology has a high rate of change.
- Computer software has become a key component of most modern systems. Software has been elevated to a highly prominent role because of its flexibility and relatively low replication cost compared with hardware.
- A significant number of software projects failed due to human issues. Physical measurement units such as the length and weight measures are challenging to apply to the software. This difficulty impacts how to plan, monitor, and control software development projects.
- Software rework to remove faults and respond to change.
- Speed and cycle time are important metrics for managing software. Software capabilities are often delivered at increasing speed to satisfy business and mission needs [13].

SEM activities occur on three levels: organizational and infrastructure management, project management, and management of the measurement program. The last two are covered in detail in this KA description. This fact does not diminish the importance of organizational and infrastructure management issues but rather points out that software organizational engineering managers should be conversant with the project management and software measurement knowledge described in this KA. They should also possess some target domain knowledge. Likewise, it also helps for managers of complex projects and programs where software is part of the system architecture to know what issues software engineering processes (versus other types of engineering processes) introduce into project management and project measurement.

Other aspects of organizational management affect software engineering — for

example, organizational policies and procedures that provide the framework for software engineering projects. These policies and procedures might need to be adjusted for effective software development and maintenance requirements. In addition, several policies specific to software engineering might need to be in place or established for the effective management of software engineering at the organizational level. For example, policies are usually necessary to establish specific organization-wide processes or procedures for software engineering tasks such as software design, software construction, estimating, monitoring and reporting. Such policies are important for effective long-term management of software engineering projects across an organization (e.g., one such policy could establish a consistent basis for analyzing past project performance and implementing improvements).

Another important aspect of organizational management is the use of personnel management policies and procedures for hiring, training and mentoring — not only for a project's success, but also for the organization's long-term success. Given the projected scarcity of skilled software engineers, it is important to provide an environment that attracts and retains good talent. Software engineering personnel can present unique training or personnel management challenges (e.g., maintaining currency in a context where the underlying technology undergoes rapid and continuous change) as part of career development.

Communication management is also often mentioned as an overlooked but important aspect of success in a field where a precise understanding of user needs, software requirements and software designs is necessary. Furthermore, portfolio management is desirable, which provides an overall view of software under development in various projects and programs (integrated projects) of planned software, and of software already in use in an organization. Also, software reuse can be a key factor in maintaining and improving productivity and competitiveness. Effective reuse requires a strategic vision

that reflects the advantages and disadvantages of reuse.

Software engineers should have a sound understanding of the aspects of management that are unique to software projects, and they should also have some knowledge of the more general aspects of management discussed in this KA (even in the first few years after graduation).

Certain attributes of organizational culture and behavior, as well as management of functional areas of the enterprise outside the immediate software engineering realm, can influence an organization's software engineering processes, albeit indirectly. Software projects are often targeted at changing the way people work — but culture change is difficult, complicated and unlikely to succeed without a significant effort. For this reason, leadership is an important attribute for program managers, as they often need to lead the charge for digital transformation. They might need to galvanize their teams and other stakeholders to bring their very best to every project pursuing major change.

Extensive information concerning project management can be found in the *Guide to the Project Management Body of Knowledge (PMBOK® Guide fifth edition)* and the *Software Extension to the PMBOK® Guide (SWX)* [1, 2]. Each of these guides includes 10 project management KAs: project integration management, project scope management, project time/schedule management, project cost management, project quality management, project resource/human management, project communications management, project risk management, project procurement management and project stakeholder management. Each KA has direct relevance to this SEM KA.

Additional information is also provided in this KA's references and list of Further Readings.

This SEM KA discusses the software project management processes shown as the first five topics in Figure 9-1 (Initiation and Scope Definition, Software Project Planning, Software Project Enactment, Review and Evaluation, Closure), as well as Software Engineering Measurement (the sixth topic

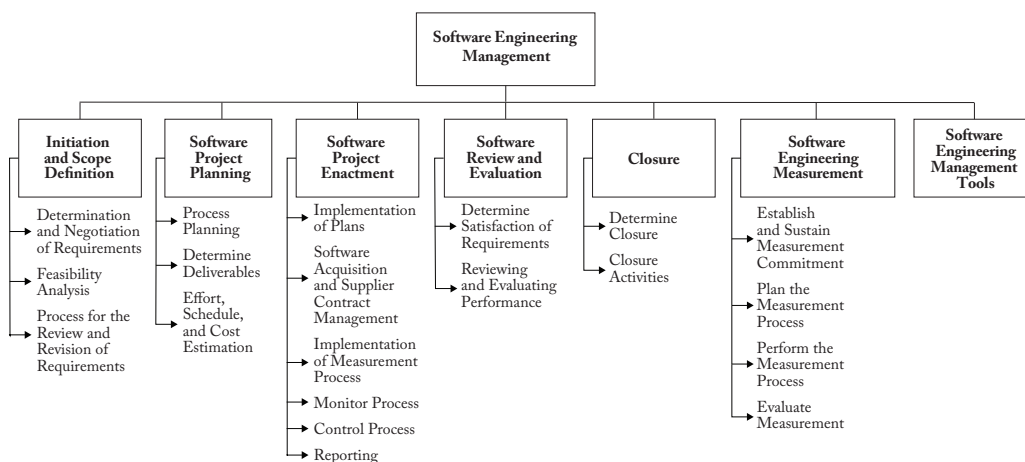


Figure 9.1. Breakdown of Topics for the Software Engineering Management KA

shown in the figure) and Software Engineering Management Tools (the seventh topic).

Unfortunately, a common perception of the software industry is that software products often are delivered late, are over budget, are of poor quality and have incomplete functionality. Measurement-informed management — a basic principle of any true engineering discipline (see Measurement in the Engineering Foundations KA) — can help improve perception and reality. In essence, management without measurement (qualitative and quantitative) suggests a lack of discipline, and measurement without management suggests a lack of purpose or context. To be effective, software engineers must use both measurement and management.

The following working definitions are adopted here:

- *Management* is a system of processes and controls required to achieve the strategic objectives set by the organization.
- *Measurement* refers to the assignment of values and labels to software engineering work products, processes and resources, plus the models derived from them, whether these models are developed using statistical or other techniques [3*, c7, c8].

The software engineering project management sections in this KA use the Software Engineering Measurement section extensively.

This KA is closely related to others in the *SWEBOK Guide*; reading the following KA descriptions will be particularly helpful in understanding this one:

- The Engineering Foundations KA describes some general measurement concepts that directly apply to the Software Engineering Measurement section of this KA. In addition, the concepts and techniques presented in the Statistical Analysis section of the Engineering Foundations KA apply directly to many topics in this KA.
- The Software Requirements KA describes activities that should be performed during the project's Initiation and Scope Definition phase.
- The Software Configuration Management KA deals with the identification, control, status accounting and auditing of software configurations, along with software release management and delivery and software configuration management tools.
- The Software Engineering Process KA describes software life cycle models and the relationships between processes and work products.

- The Software Quality KA emphasizes quality as a management goal and as an aim of many software engineering activities.
- The Software Engineering Economics KA discusses how to make software-related decisions in a business context.

BREAKDOWN OF TOPICS FOR SOFTWARE ENGINEERING MANAGEMENT

Because most software development life cycle (SDLC) models require similar activities that may be executed in different ways, the topic breakdown, shown in Figure 9-1, is activity-based. The top-level elements shown in the figure are activities that are usually performed when a software development project is being managed, regardless of which SDLC model is being used (see Software Life Cycle Models in the Software Engineering Process KA). This breakdown does not recommend a specific life cycle model. However, it is important to note the choice of the SDLC can have a impact on program activities to accommodate changing requirements.

Delivery speed, continuous adaptation and frequent modular upgrades to deliver new capabilities are often key business differentiators and project management imperative [11, 13]. These imperatives should be balanced with risk management activities.

Several software life cycle process models have been and are being developed to shorten development cycles in response to changing business needs, specifically, changing software requirements. Most of these processes involve Agile SDLC approaches [14]. The Agile approach assumes that teams can develop high-quality, adaptive software using continuous design improvement principles and testing based on rapid feedback and change. In comparison, the traditional approach assumes that software-intensive systems are fully specifiable and predictable and can be built through meticulous and extensive planning. The management style associated with the Agile approach emphasizes

leadership and collaboration at the team level, whereas the management style of the highly predictive approach is more formal (top-down). Many Agile approaches integrate different management approaches.

For example, DevSec/Ops is a culture and an Agile approach to modern software delivery that aligns development (Dev), security (Sec) and operations (Ops) groups into an integrated team focused on continuous, incremental delivery of capabilities. The main characteristic of DevSec/Ops is that this approach automates, continuously monitors and applies security at all phases of the software life cycle: plan, develop, build, test, release, deliver, deploy, operate and monitor. In DevSec/Ops, testing and security are shifted to the left through automated unit, functional, integration and security testing. This is a key DevSec/Ops differentiator; security/quality assurance (QA) and other nonfunctional and functional capabilities are tested and built simultaneously [11, 14]. Whereas DevSec/Ops encompasses the culture and processes that enable rapid, continual delivery of cyber-resilient systems, complex software-embedded systems can have additional demands that must also be integrated into the DevSec/Ops culture and processes, such as safety. Elevating these demands to be on par with DevSec/Ops highlights the importance of incorporating quality into all program aspects. The complexity of the end-to-end DevSecOps tools and of using emerging technologies such as artificial intelligence (AI) and machine learning (ML) to leverage those tools adds another dimension [15]. For example, Agile and DevOps approaches are reasonably well-established, but in case of AI-based software, new SLDCs maybe required to manage the complexity brought by AI to the software.

It is important to understand the difference between *phases* and *activities* and why an activities breakdown is used. The Project Management Institute (PMI) describes a *phase* this way: “The completion and approval of one or more deliverables characterizes a project phase.” A *deliverable* is a measurable, verifiable work product such as a specification,

feasibility study report, detailed design document or working prototype. Some deliverables correspond to part of the project management process, whereas others are the end products or components of the end products for which the project was conceived. The deliverables, and hence the phases, are part of a generally sequential process designed to ensure proper control of the project and to attain the desired product or service, which is the project's objective. From a project management perspective, phases help accomplish project objectives and maintain control over the project.

The activity-based breakdown in Figure 9-1 shows what happens but does not imply when, how or how many times each activity occurs. The seven topics are the following:

- Initiation and Scope Definition, which deals with the decision to embark on a software engineering project
- Software Project Planning, which addresses the activities undertaken to prepare for a successful software engineering project from the management perspective
- Software Project Enactment, which deals with generally accepted SEM activities that occur during a software engineering project's execution
- Review and Evaluation, which deals with ensuring that technical, schedule, cost and quality engineering activities are satisfactory
- Closure, which addresses the activities accomplished to complete a project
- Software Engineering Measurement, which deals with the effective development and implementation of measurement programs in software engineering organizations
- Software Engineering Management Tools, which describes the selection and use of tools for managing a software engineering project

1. Initiation and Scope Definition

Project initiation focuses on reviewing the software requirements and determining the

need, scope, feasibility, and authorization for a software project. Once project feasibility has been established, the remaining tasks in this section are specifying the software requirements and selecting the processes for requirements revision and review.

1.1. Determination and Negotiation of Requirements [3*, c3]

Determining and negotiating the project requirements are the overarching goals of the tasks undertaken during this phase (see the Software Architecture KA and Software Requirements KA). Activities should include software requirements review (e.g., elicitation, analysis, specification, and validation). Methods and techniques should be selected and applied considering the various stakeholder perspectives. Requirements provide the basis for all that follows on a software project and are captured in a Project Charter or other high-level project initiation document.

1.2. Feasibility Analysis [4*, c5]

The purpose of the feasibility analysis is to develop a clear description of project objectives and to evaluate alternative approaches to determine whether the proposed project solution is the best approach, given the constraints of technology, resources, finances and changes to ethical, environmental, and socio-technical considerations. An initial project and product scope statement, project deliverables, project duration constraints, and an estimation of resources needed should be prepared.

Resources (which can be internal or external to the organization) include infrastructure, support, and people with the necessary core competencies. The feasibility analysis often requires estimations of effort and cost based on appropriate methods. (See Section 2.3, Effort, Schedule and Cost Estimation.)

An initial work breakdown structure (WBS) and context diagram may be developed during the project's Initiation and Scope Definition phase activities. Breaking work into smaller tasks is a common productivity

technique that makes the work more manageable and approachable. As the project tool that uses this technique, WBS is an important project management document. While a WBS can be used to organize cost and schedule tracking, the WBS does not itself include cost and schedule baselines. Schedules are developed as part of the next activity, project planning (section 2).

An engineering context diagram defines the boundary between the system (or a part of the system) and its environment, showing the entities interacting with it. This document is important in defining management and technical interfaces and trade-offs that must be considered [1]. While engineers are developing the WBS, they should consider all configuration items as tasks to have under control.

1.3. *Process for the Review and Revision of Requirements* [3*, c3]

Given the inevitability of change, stakeholders should agree on how requirements and scope will be reviewed and revised (e.g., change management and trade-off procedures, iterative cycle retrospectives). (See the Requirements KA.) This indicates that scope and requirements will not be “set in stone” but can and should be revisited at predetermined points as the project unfolds (for example, at the time when backlog priorities are created or at milestone reviews). If changes are accepted, then forward or backward traceability analysis and risk analysis should be used to ascertain the impact of those changes. For example, backward traceability may link the test script to its associated requirement and design. This link helps monitor the status of requirements satisfaction and helps make decisions to stop testing. It also helps in making tradeoffs regarding requirements and design. (See Section 2.5, Risk Management, and Software Configuration Control in the Software Configuration Management KA.)

A managed-change approach can also form the basis for evaluating success during closure of an incremental cycle or an entire project,

based on changes that occurred along the way. (See Topic 5, Closure).

2. **Software Project Planning**

A key step in software project planning should be selecting an appropriate SDLC model and, perhaps, tailoring it based on project scope, software requirements and a risk assessment. The *SWX* [2] states that project life cycles occupy a continuum from predictive to adaptive. Factors that characterize the positions of software project life cycles within the continuum include (but are not limited to) the various ways requirements and plans are handled, how risk and cost are managed, and key stakeholder involvement. Highly predictive software project life cycles emphasize requirements specification and detailed planning during the project’s initiation and planning phases. Detailed plans based on a known architecture, requirements and constraints are used to reduce risk and cost. Milestones are planned, versus continuous key stakeholder involvement. Highly adaptive software project life cycles, on the other hand, are characterized by progressive requirements specification based on short iterative development cycles. Risk and cost are reduced by progressive evolution of initial plans, and key stakeholders are continuously involved [2].

Other factors to consider include the nature of the application domain, functional and technical complexity, and software quality requirements. (See Software Quality Requirements in the Software Quality KA.)

In all SDLCs, risk assessment should be an element of initial project planning, and the “risk profile” of the project should be discussed and accepted by all relevant stakeholders. Software quality management processes (see Software Quality Management Processes in the Software Quality KA) should be planned along with project planning. This planning should establish procedures and responsibilities for software quality assurance (SQA), verification and validation, reviews, and audits. (See the Software Quality KA.) Processes and responsibilities

for ongoing review and revision of the project plan and related plans should also be clearly stated and agreed upon.

2.1. *Process Planning*

[3*, c3, c4, c5], [5*, c1]

SDLC models span a continuum from predictive to adaptive. (See Software Life Cycle Models in the Software Engineering Process KA.) Predictive SDLCs are characterized by the development of detailed software architecture and software requirements, detailed project planning, and minimal planning for iteration among development phases. Adaptive SDLCs are designed to accommodate emergent software requirements and iterative adjustment of plans. A highly predictive SDLC executes the first five processes listed in Figure 9-1 in a linear sequence with revisions to earlier phases only as necessary. Adaptive SDLCs are characterized by iterative development cycles. SDLCs in the midrange of the SDLC continuum produce increments of functionality on either a preplanned schedule (on the predictive side of the continuum) or as the products of frequently updated development cycles (on the adaptive side of the continuum).

Well-known SDLCs include the waterfall, incremental and spiral models, plus various Agile software development approaches [2, 11] [3*, c2].

Relevant methods (see the Software Engineering Models and Methods KA) and tools should be selected as part of planning. Automated tools that will be used throughout the project should also be planned for and acquired. Tools might include those for project scheduling, software requirements, software design, software construction, software maintenance, software configuration management, software engineering process and software quality, among others. Many of these tools should be selected based primarily on the technical considerations discussed in other KAs, but some of those concerns are closely related to the management considerations discussed in this chapter.

2.2. *Determine Deliverables*

[3*, c4, c5, c6]

Each project activity's work product(s) (e.g., software architecture design documents, inspection reports, tested software) should be identified and characterized. Opportunities to reuse software components from previous projects or to use off-the-shelf software products should be evaluated. Software procurement and use of third parties to develop deliverables should be planned and suppliers selected. (See Section 3.2, Software Acquisition and Supplier Contract Management.)

2.3. *Effort, Schedule, and Cost Estimation*

The topic of estimation in general is addressed in the Software Engineering Economics KA. Questions like "What is estimation?" and "Why do we estimate?" are addressed there. This section addresses management-specific estimation topics.

Estimating costs for software projects is an error-prone process. The effort required for any given software project depends almost entirely on human elements: individuals' experience and capabilities, team members' interactions, and the culture of the software development environment. Dynamic environmental factors, such as rapid technology evolution, changing and emergent requirements, and the intangible nature of the product, also significantly affect cost management. Estimating costs when this much variability exists is difficult even when significant historical data exists. Software project managers should use multiple estimation approaches and then reconcile the differences among the estimates [3, 10, 11].

When data is available, the estimated range of effort required for a project, or parts of a project, can be determined using a calibrated estimation model based on historical size and effort data. It is best to also use bottom-up estimation techniques based on estimates from those who will accomplish the work and historical data based on similar projects [2]. Task dependencies can be established, and potential opportunities for completing tasks

concurrently and sequentially can be identified and documented, using a Gantt chart, for example. In predictive SDLC projects, the expected schedule of tasks, with projected start times, durations and end times, is typically produced during planning. In adaptive SDLC projects, an overall estimate of effort and schedule is typically developed from the initial understanding of the requirements, or, alternatively, constraints on overall effort and schedule may be specified and used to determine an initial estimate of the number of iterative cycles and estimates of effort and other resources allocated to each cycle.

Resource requirements (for example, people and tools needed) can usually be translated into cost estimates. The estimation of effort, schedule and cost is an iterative activity that should be negotiated and revised among affected stakeholders until consensus is reached on resources and time available for project completion. Program managers often use a model that links four association role types: responsible, accountable, consulted, and informed (i.e., RACI) to facilitate this process. Responsible roles produce deliverables; accountable roles check the deliverables; consulted roles advise on tasks; and informed roles are kept informed throughout these processes. Project managers should constantly monitor stakeholder requirements and changes as they evolve to analyze their impact on the project cost and schedule. This is usually more important in Agile software development projects, where stakeholder requirements are dynamic because changes might occur rapidly as the project progresses.

2.4. Resource Allocation [3*, c5, c10, c11]

Equipment, facilities and people should be allocated to the identified tasks, including allocating responsibilities for completing various project elements and the overall project. A matrix that shows who is responsible for, accountable for, consulted about and informed about each task can be used. Resource allocation is based on and constrained by the availability of resources and their optimal

use, and by issues relating to personnel (e.g., productivity of individuals and teams, team dynamics, and team structures).

2.5. Risk Management [3*, c9] [5*, c5]

Risk and uncertainty are related but distinct concepts. Uncertainty results from a lack of information. Risk is effect of uncertainty on objectives that has negative (threats) or positive (opportunities) consequences on objectives.

Risk management entails identifying risk factors, analyzing probability and potential impact of each risk factor, prioritizing risk factors, and developing risk mitigation strategies to reduce the probability of a negative event and to minimize the negative impact if a risk factor becomes a problem. Risk management data can be used to represent the project risk profile; this data is often part of a risk register. A risk register is a document used as a risk management tool. It can be used to fulfill regulatory compliance, serving as a repository for all risks identified and for additional information about each risk [2]. Risk assessment methods (e.g., expert judgment, historical data, decision trees and process simulations) can sometimes be used to identify and evaluate risk factors.

Project abandonment conditions can also be determined with all relevant stakeholders. Software-unique aspects of risk, such as software engineers' tendency to add unneeded features or the risks related to software's intangible nature, can influence risk management for software projects. Particular attention should be paid to managing risks related to software quality requirements such as safety or security [11]. (See the Software Quality KA.) Risk management should be done not only at the beginning of a project, but also at periodic intervals throughout the project life cycle.

2.6. Quality Management [3*, c4] [4*, c2]

According to the *PMBOK® Guide*, Project quality management includes the performing

organization's processes and activities that determine quality policies, objectives and responsibilities so the project will satisfy the needs for which it was undertaken. This section discusses additional considerations for managing software project quality [1]. Software quality requirements for a software project and the associated work products should be identified, perhaps both quantitatively and qualitatively. Quality attributes of software include but are not limited to safety, security, reliability, availability, performance, ease of use and ease of modification. *SWX* Section 1.9 lists quality attributes that are important for software users (e.g., efficiency, safety, security, reliability, availability) and quality attributes that are important to software developers and maintainers (e.g., maintainability is important to those who provide sustainment services) [1]. ISO/IEC 25000 series of standards provides extensive lists of software quality attributes that align with different stakeholder needs [2]. This alignment is consistent with ISO/IEC/IEEE 15939 and practical software and systems measurement (PSM) [2, 9.11].

Large portions of system functionality are shifting from hardware to software to capitalize on the increased flexibility and speed of component delivery that software can provide. However, with these benefits come other challenges — for example, the need for increased management of software quality requirements (e.g., cybersecurity) throughout the SDLC [11]. Thresholds for acceptable quality measurements should be set for each software quality requirement based on stakeholder needs and expectations. Procedures concerned with ongoing SQA and quality improvement throughout the development process and with verifying and validating the deliverable software product should also be specified during quality planning (e.g., technical reviews and inspections or demonstrations of completed functionality). (See the Software Quality KA.)

2.7. Plan Management

[3*, c4]

Except for older predictive programs, documenting and managing formal plans are becoming less emphasized in managing most software projects. (e.g., documentation plans are rarely used, especially when Model-Based Systems Engineering (MBSE) is used for product data). The said, where they are used, plans should be developed and managed for software projects when change is expected. The magnitude of the planning effort and the plan's content should be determined partly by the risk of not developing the plan. The management of the project plan should itself be planned. Plans and processes selected for software development should be systematically monitored, reviewed, reported and, when appropriate, revised. Plans associated with supporting processes (e.g., documentation, software configuration management, and problem resolution) also should be managed. Reporting, monitoring and controlling a project should fit within the selected SDLC and the realities of the project. Plans should account for the various artifacts that will be used to manage the project.

Project managers of predictive life cycle software projects put substantial effort into up-front development of the project plan and integration of subsidiary plans developed by support personnel from other organizational units (e.g., estimation specialists in the Project Management Office (PMO)).

In other types of programs (e.g., adaptive programs) where formal plans are not usually used, the emphasis should be on selecting and retaining project information useful in project control and future projects, and establishing strategy, policies, and procedures. For example, in adaptive programs, managers will usually spend less effort up front on developing detailed scope, cost and schedule plans. But significant effort is typically spent defining monitor and control processes, such as requirements traceability, to ensure coordination among the project members or teams as the emerging plans are implemented [2].

3. Software Project Execution

During software project enactment (also known as *project execution*), plans are implemented, and the processes embodied in the plans are enacted. Throughout, there should be a focus on adherence to the selected SDLC processes, with an overriding expectation that adherence will satisfy stakeholder requirements and achieve the project's objectives. Fundamental to enactment are the ongoing management activities of monitoring, controlling and reporting.

3.1. *Implementation of Plans* [4*, c2]

Project activities should follow the project plan and supporting plans. Project activities use resources (personnel, technology and funding) and generate work products (software design, software code and software test cases).

3.2. *Software Acquisition and Supplier Contract Management* [3*, c3, c4]

Software acquisition and supplier contract management concern issues involved in contracting with customers of the software development organization who acquire the deliverable work products and with suppliers who supply products or services to the software engineering organization.

Software acquisition is common practice in software development projects, with integrated development environments (IDEs) and package libraries allowing software engineers to acquire third-party libraries with minimal steps, facilitating the assessment of risk, legality and suitability. However, software is no longer exclusively acquired as a shrink-wrapped product via a complex supply chain process and purchasing route. The ease of acquiring software has resulted in a common attack surface and led to security vulnerabilities. Organizations should consider introducing technical or procedural controls to minimize risk potentially exposed by unfiltered access to external library repositories.

The different software acquisition classes include commercial off-the-shelf (COTS) software — an existing product acquired “as is” from another software vendor, with applicable license terms; software developed exclusively for the organization by another party — typically contracted and sometimes a customization of COTS software; open source software — nominally free, although the organization may purchase enhanced support or maintenance and must review the license for restrictions on use; customer loaned software — typically to provide simulation or integration with another system element; software as a service (SaaS) — which might include software the organization rents to fulfill a particular need (for example, a cloud-based hosting, source control or development environment).

Software projects typically use different acquisition approaches to obtain the necessary software components. However, regardless of how the software components are obtained, the following activities should be performed: verifying that each component is complete, correct and consistent concerning the architectural design and software requirements for that component; integrating the components; verifying that the integrated components are correct, complete and consistent concerning the architectural design and the software requirements; and validating that the integrated components will satisfy their intended purpose when used in their intended operating environment.

Different acquisition approaches (for obtaining software components) require different approaches to managing the project. For example, custom development requires detailed planning for the numbers and skills of the software developers, organizing the development team(s), allocating requirements to the teams, specifying project metrics to be collected, monitoring progress, and applying corrective actions when actual progress does not agree with planned progress. Licensing components involves evaluating candidate components; selecting appropriate components; and negotiating terms, conditions, and delivery dates for the selected components.

This might involve selecting appropriate contracts, such as fixed price, time-and-materials, cost plus fixed fee, and cost-plus incentive fee. Agreements with customers and suppliers typically specify the scope of work and the deliverables. The agreements can also include special clauses, such as clauses establishing penalties for late delivery or no delivery, and intellectual property agreements that specify what the suppliers are providing and what the acquirer is paying for, plus what will be delivered to and owned by the acquirer. For software developed by suppliers (both those internal to and those external to the software development organization), agreements commonly establish software quality requirements.

In software contracting with suppliers, data set acquisition is usually important. It includes the process of obtaining specific datasets from external vendors or partners as part of a software development project or service agreement. This can occur in various scenarios, such as: data licensing agreements, data provisioning, custom data acquisition and data integration services.

After the agreement has been put in place, executing the project in compliance with the terms of the agreement should be managed. (See Chapter 12, *Software Extension to the PMBOK® Guide (SWX)*, Software Procurement Management, for more information on this topic [2].)

3.3. Implementation of Measurement Process [3*, c7]

The measurement process should be enacted during the software project to ensure that relevant and useful data is collected. (See Sections 6.2, Plan the Measurement Process, and 6.3, Perform the Measurement Process.)

3.4. Monitor Process [3*, c8]

Adherence to the scope, project plan and related plans should be assessed continually and at predetermined intervals. Outputs and completion criteria for each task should also be assessed. Deliverables should be evaluated

for their required characteristics (for example, via inspections or by demonstrating working functionality). Effort expenditure, schedule adherence, costs to date, and resource use should be analyzed. The project risk profile (see Section 2.5, Risk Management) should be revisited, and adherence to software quality requirements should be evaluated (see Software Quality Requirements in the Software Quality KA).

Measurement data should be analyzed. (See Statistical Analysis in the Engineering Foundations KA.) Variance analysis should be conducted to determine deviation of actual from expected outcomes and values. This analysis might examine cost overruns, schedule slippage or other measures. Outlier identification and analysis of quality and other measurement data should be performed (e.g., defect analysis). (See Software Quality Measurement in the Software Quality KA.) Risk exposures should be recalculated. (See Section 2.5, Risk Management.) These activities can enable problem detection and exception identification based on thresholds that have been exceeded. Outcomes should be reported as necessary or when thresholds have been exceeded. For example, the timely identification, mitigation, and resolution of software security vulnerabilities and weaknesses that exceed expectations can affect the system's security posture [11].

3.5. Control Process [3*, c7, c8]

Project monitoring activities provide the basis for making decisions. Where appropriate, and when the probability and impact of risk factors are understood, changes can be made to the project. This may take the form of corrective action (e.g., retesting certain software components). It might involve incorporating additional actions (e.g., deciding to use prototyping to assist in software requirements validation; see Prototyping in the Software Requirements KA). It might also entail revising the project plan and other project documents (e.g., the software requirements specification) to accommodate unanticipated events and their implications.

In some instances, the control process might lead to abandonment of the project. In all cases, the software development team should adhere to software configuration control and software configuration management procedures. (See the Software Configuration Management KA.) Decisions should be documented and communicated to all relevant parties, plans should be revisited and revised when necessary, and relevant data should be recorded. (See Section 6.3, Perform the Measurement Process.)

3.6. *Reporting* [3*, c11]

Progress to date should be reported at specified and agreed-upon times both within the organization (e.g., to a project steering committee) and to external stakeholders (e.g., clients or users). Reports should focus on the information needs of the target audience as opposed to the detailed status reporting within the project team.

4. **Software Review and Evaluation**

At prespecified times and as needed, overall progress toward the stated objectives and satisfaction of stakeholder (user and customer) requirements should be evaluated. Similarly, assessments of the effectiveness of the software process, the personnel involved, and the tools and methods used should also be undertaken regularly and as circumstances demand.

4.1. *Determining Satisfaction of Requirements* [4*, c8]

Achieving stakeholder satisfaction is a principal goal of the software engineering manager. Progress toward this goal should be assessed periodically. Progress should be assessed upon achieving a major project milestone (e.g., completing software design architecture or completing a software technical review) or upon completion of an iterative development cycle that results in a product increment. Variances from software

requirements should be identified, and appropriate actions should be taken.

As in the control process activity above (see Section 3.5, Control Process), software configuration control and software configuration management procedures should be followed (see the Software Configuration Management KA). Decisions should be documented and communicated to all relevant parties; plans should be revisited and revised as necessary; and relevant data should be recorded (see Section 6.3, Perform the Measurement Process).

4.2. *Reviewing and Evaluating Performance* [3*, c8, c10]

Periodic performance reviews for project personnel can provide insight into the likelihood of adherence to plans and processes and possible areas of difficulty (e.g., team member conflicts). The various project methods, tools and techniques should be evaluated for effectiveness and appropriateness. The project's process should also be systematically and periodically assessed for relevance, utility and efficacy. Where appropriate, project changes should be made and managed.

5. **Closure**

An entire project, a major project phase or an iterative development cycle reaches closure when all the plans and processes have been enacted and completed. The criteria for project, phase or iteration success should then be evaluated. Once closure has been established, archival, retrospective and process improvement activities can be performed.

5.1. *Determining Closure* [1, s3.7, s4.6]

Closure occurs when the specified tasks for a project, a phase or an iteration have been completed and satisfactory achievement of the completion criteria has been confirmed. Software requirements can be confirmed as satisfied or not, and the degree of achieving the objectives can be determined. Closure

processes should involve relevant stakeholders and document relevant stakeholders' acceptance; any known problems should be documented.

5.2. Closure Activities [2, s3.7, s4.8]

After closure has been confirmed, project materials should be archived in accordance with stakeholder agreed-upon rules for archival methods, location and duration — possibly including destruction of sensitive information, software and the medium on which copies are resident. For example, these rules could require that during closure, all data is removed and destroyed from any devices that contain relevant information before physical disposal of the devices (e.g., the hard drives of personal computers, servers, mainframes, personal digital assistants (PDAs), routers, firewalls, switches, tapes, diskettes, CDs, DVDs, cell phones, printers, universal serial bus (USB) data storage devices).

The organization's measurement database should be updated with relevant project data. A project, phase or iteration retrospective analysis should be undertaken so that issues, problems, risks and opportunities encountered can be analyzed. (See Topic 4, Review and Evaluation.) Lessons learned should be drawn from the project and fed into organizational learning and improvement endeavors.

6. Software Engineering Measurement

The importance of software engineering measurement for good management and engineering practices is widely acknowledged. (See Measurement in the Engineering Foundations KA.) Effective software engineering measurement has become one of the cornerstones of organizational maturity. Measurement can be applied to organizations, projects, processes and work products. This section focuses on applying measurement at the levels of projects, processes and work products.

This section follows ISO/IEC/IEEE 15939 standard [6], which describes a process to define the activities and tasks necessary to implement a software measurement process. The standard also includes a measurement information model. This model in the PSM Continuous Iterative Development Measurement Framework report is also elaborated for SDLC approaches [9].

6.1. Establish and Sustain Measurement Commitment [7*, c1, c2]¹

- Establish measurement requirements. Each measurement endeavor should be guided by organizational objectives and driven by a set of measurement requirements established by the organization and the project (e.g., an organizational objective might be first to market).
- Establish the scope of measurement. The project team should establish the organizational unit to which each measurement requirement is to be applied. This might be a functional area, a single project, a single site or an entire enterprise. The temporal scope of the measurement effort should also be considered because the time series of some measurements might be required (e.g., to calibrate estimation models). (See Section 2.3, Effort, Schedule and Cost Estimation.)
- Establish the team's commitment to measurement. The commitment should be formally established, communicated and supported by resources.
- Commit measurement resources. An organization's commitment to measurement is an essential factor for success, as evidenced by the assignment of resources for implementing the measurement process. Assigning resources includes allocation of responsibility for the various tasks of the measurement process (such as analyst and librarian). Adequate funding, training, tools and

¹ These two chapters can be downloaded free of charge from <http://www.psmc.com/PSMBook.asp>.

support to conduct the process should also be allocated.

6.2. *Plan the Measurement Process*

[7*, c1, c2]¹

- Characterize the organizational unit. The organizational unit provides the context for measurement, so the organizational context should be explicit, including the organization's constraints on the measurement process. The characterization can be stated in terms of organizational processes, application domains, technology, organizational interfaces and organizational structure.
- Identify information needs. Information needs are based on the organizational unit's goals, constraints, risks, and problems and may be derived from business, organizational, regulatory and/or product objectives. Stakeholders should identify, prioritize, document, communicate and review these needs.
- Select measures. Select candidate measures, with clear links to the information needs. Select measures based on the priorities of the information needs and other criteria such as cost of collection; degree of process disruption during collection; ease of obtaining accurate, consistent data; and ease of analysis and reporting. Internal quality characteristics (see Models and Quality Characteristics in the Software Quality KA) are often not contained in the contractually binding software requirements. Therefore, consider measuring the software's internal quality to provide an early indicator of potential issues that might affect external stakeholders.
- Define data collection, analysis and reporting procedures. This encompasses collection procedures and schedules, storage, verification, analysis, reporting and data configuration management.
- Select criteria for evaluating the information products. The organizational unit's technical and business objectives

influence evaluation criteria. Information products include those associated with the product produced and those associated with the processes used to manage and measure the project.

- Provide resources for measurement tasks. The appropriate stakeholders should review and approve the measurement plan to include all data collection procedures; storage, analysis, and reporting procedures; evaluation criteria; schedules; and responsibilities. Criteria for reviewing these artifacts should be established at the organizational unit level or higher and should be used as the basis for these reviews. Such criteria should consider experience, resource availability and potential disruptions to projects when changes from current practices are proposed. Approval demonstrates commitment to the measurement process.
- Identify resources to be made available for implementing the planned and approved measurement tasks. Resource availability may be staged in cases where changes are piloted before widespread deployment. Consider the resources necessary for successful deployment of new procedures or measures.
- Acquire and deploy supporting technologies. This includes evaluating available supporting technologies, selecting the most appropriate technologies, acquiring those technologies and deploying those technologies.

6.3. *Perform the Measurement Process*

[7*, c1, c2]

Integrate measurement procedures with relevant software processes. The measurement procedures, such as data collection, should be integrated into the software processes they measure. This might involve changing current software processes to accommodate data collection or generation activities. It might also involve analyzing current software processes to minimize additional effort and evaluating

the effect on employees to ensure acceptance of the measurement procedures. Consider morale issues and other human factors. In addition, communicate the measurement procedures to those providing the data. Training and support might also be needed. Data analysis and reporting procedures are typically integrated similarly into organizational and project processes.

Collect data. Measurement data should be collected and analyzed. Data should be collected, verified and stored. Collection can sometimes be automated by using SEM tools (see Topic 7, Software Engineering Management Tools) to analyze data and develop reports. Data may be aggregated, transformed or recorded as part of the analysis, using a degree of rigor appropriate to the nature of the data and the information needs. This analysis typically produces graphs, numbers or other indicators that inform conclusions and recommendations to present to stakeholders. (See Statistical Analysis in the Engineering Foundations KA.) The results and conclusions are reviewed using the organization's formal or informal process. Data providers and measurement users should participate in reviewing the data to ensure it is meaningful and accurate and can result in reasonable actions.

Communicate results. Document and communicate information products to users and stakeholders.

6.4. Evaluate Measurement [7*, c1, c2]

Evaluate information products and the measurement process against specified evaluation criteria, and determine the strengths and weaknesses of the information products or process. An internal process or an external audit can be used to perform the evaluation, including feedback from measurement users. Record lessons learned in an appropriate database.

Identify potential improvements. Such improvements might be changes in the format of indicators, changes in units measured or reclassification of measurement categories. Determine potential improvements' costs

and benefits, and report appropriate improvement actions.

Communicate proposed improvements to the measurement process owner and stakeholders for review and approval. Also, communicate the lack of potential improvements if the analysis fails to identify any.

7. Software Engineering Management Tools [3*, c5, c6, c7]

SEM tools are often used to provide visibility and control of SEM processes. Some tools are automated, whereas others are manually implemented. In addition, there has been a recent trend toward using integrated suites of software engineering tools throughout a project to plan, collect and record, monitor and control, and report project and product information. Tools can be divided into the following categories:

Project planning and tracking tools. Project planning and tracking tools can be used to estimate project effort and cost and to prepare project schedules. For example, some projects use automated estimation tools that use a software product's estimated size and other characteristics as input and then estimate the required total effort, schedule and cost. Planning tools also include automated scheduling tools that analyze the WBS tasks, their estimated durations, their precedence relationships and the resources assigned to each task to produce a Gantt chart.

Tracking tools can be used to track project milestones, regularly scheduled project status meetings, scheduled iteration cycles, product demonstrations and action items.

Risk management tools. Risk management tools (see Section 2.5, Risk Management) can be used to track risk identification, analysis and monitoring. These tools include simulation or decision trees to analyze the effect of costs versus payoffs and subjective estimates of the probabilities of risk events. For example, Monte Carlo simulation tools can be used to produce probability distributions of effort, schedule and risk by algorithmically combining multiple input probability distributions.

Communication tools. Communication tools can help provide timely and consistent information to relevant stakeholders involved in a project. Examples of such tools are email notifications and broadcasts to team members and stakeholders; regular communications of meeting minutes; and charts showing progress, backlogs, and maintenance request resolutions.

Measurement tools. Measurement tools support activities related to the software measurement program. (See Topic 6, Software Engineering Measurement.) There are few completely automated tools in this category. Measurement tools to gather, analyze and report project measurement data may be based on spreadsheets developed by project team members or organizational employees.

MATRIX OF TOPICS VS. REFERENCE MATERIAL

	Fairley 2009 [3*]	Sommerville 2016 [4*]	Boehm and Turner 2003 [5*]	McGarry et al. 2001 [7*]
1. Initiation and Scope Definition				
<i>1.1. Determination and Negotiation of Requirements</i>	c3			
<i>1.2. Feasibility Analysis</i>		c4		
<i>1.3. Process for the Review and Revision of Requirements</i>	c3			
2. Software Project Planning				
<i>2.1. Process Planning</i>	c2, c3, c4, c5		c1	
<i>2.2. Determine Deliverables</i>	c4, c5, c6			
<i>2.3. Effort, Schedule and Cost Estimation</i>	c6			
<i>2.4. Resource Allocation</i>	c5, c10, c11			
<i>2.5. Risk Management</i>	c9		c5	
<i>2.6. Quality Management</i>	c4	c24		
<i>2.7. Plan Management</i>	c4			
3. Software Project Enactment				
<i>3.1. Implementation of Plans</i>		c2		
<i>3.2. Software Acquisition and Supplier Contract Management</i>	c3, c4			
<i>3.3. Implementation of Measurement Process</i>	c7			
<i>3.4. Monitor Process</i>	c8			
<i>3.5. Control Process</i>	c7, c8			
<i>3.6. Reporting</i>	c11			
4. Review and Evaluation				
<i>4.1. Determining Satisfaction of Requirements</i>				

4.2. <i>Reviewing and Evaluating Performance</i>	c8, c10			
5. Closure				
5.1. <i>Determining Closure</i>				
5.2. <i>Closure Activities</i>				
6. Software Engineering Measurement				
6.1. <i>Establish and Sustain Measurement Commitment</i>				c1, c2
6.2. <i>Plan the Measurement Process</i>				c1, c2
6.3. <i>Perform the Measurement Process</i>				c1, c2
6.4. <i>Evaluate Measurement</i>				c1, c2
7. Software Engineering Management Tools	c5, c6, c7			

FURTHER READINGS

A Guide to the Project Management Body of Knowledge (PMBOK® Guide) [1].

The *PMBOK® Guide* provides guidelines for managing individual projects and defines project management-related concepts. It also describes the project management life cycle and its related processes, and the project life cycle. It is a globally recognized guide for the project management profession.

Software Extension to the Project Management Body of Knowledge (PMBOK®) Guide [2].

SWX provides adaptations of and extensions to the generic practices of project management documented in the *PMBOK® Guide* for managing software projects. The primary contribution of this extension to the *PMBOK® Guide* is a description of processes for managing adaptive life cycle software projects.

IEEE Standard Adoption of ISO/IEC 15939 [6].

This international standard identifies a process that supports defining suitable measures

to address specific information needs. It identifies the activities and tasks necessary to successfully identify, define, select, apply and improve measurement within an overall project or organizational measurement structure.

J. McDonald, *Managing the Development of Software Intensive Systems*, Wiley, 2010 [8].

This textbook introduces project management for beginning software and hardware developers, plus unique advanced material for experienced project managers. Case studies are included for planning and managing verification and validation for large software projects and complex software and hardware systems, as well as inspection results and testing metrics to monitor project status.

REFERENCES

- [1] Project Management Institute, *A Guide to the Project Management Body of Knowledge (PMBOK® Guide)*, 6th ed., Newton Square, PA: Project Management Institute, 2017.

- [2] *Software Extension to the Project Management Body of Knowledge (PMBOK® Guide)*, Fifth Edition, Project Management Institute, 2013.
- [3*] R. E. Fairley, *Managing and Leading Software Projects*. Hoboken, NJ: Wiley IEEE Computer Society Press, 2009.
- [4*] I. Sommerville, *Software Engineering*, 10th ed., New York: Addison-Wesley, 2016.
- [5*] B. Boehm and R. Turner, *Balancing Agility and Discipline: A Guide for the Perplexed*. Boston: Addison-Wesley, 2003.
- [6] IEEE, *IEEE Standard Adoption of ISO/IEC 15939: 2007 Systems and Software Engineering Measurement Process*, ed: IEEE, 2017.
- [7*] J. McGarry et al., *Practical Software Measurement: Objective Information for Decision Makers*, Addison-Wesley Professional, 2001.
- [8] J. McDonald, *Managing the Development of Software-Intensive Systems*. Hoboken, NJ: John Wiley and Sons, Inc., 2010.
- [9] *Practical Software and Systems Measurement Continuous Iterative Development Measurement Framework Parts 1–3: Concepts, Definitions, Principles, and Measures*, Version 2.1, 15 April 2021.
- [10] S. Sheard, M. Bouyaud, M. Osaisai, J. Siviy, and K. Nidiffer, “Book Club” *Guides a Working Group to Create INCOSE System-Software Interface Products, INSIGHT*, Volume 24, Issue 2, 2021.
- [11] K. Nidiffer, C. Woody, and T.A. Chick, *Program Manager’s Guidebook for Software Assurance, Special Report*, CMU/SEI-2018-SR-025, Software Solutions and CERT Divisions, Software Engineering Institute/ Carnegie Mellon University, August 2018.
- [12] R.E. Fairley, *Systems Engineering of Software-Enabled Systems*, ISBN 978-1-119-53501-0, 2019.
- [13] Defense Innovation Board, *Software Is Never Done: Refactoring the Acquisition Code for Competitive Advantage Defense*, v3.3, March 12, 2019.
- [14] “DevOps: Building Reliable and Secure Systems Including Application Build, Package, and Deployment,” *IEEE Standard*, 2675-2021, 2021.
- [15] M. Chemuturi and T. Cagley, *Mastering Software Project Management: Best Practices, Tools and Techniques*, J. Ross Publishing, July 2010.